DEBRE TABOR UNIVERSITY

Gafat Institute of Technology

Department of Computer Science

INTERNSHIP REPORT

Development of a Pharmacy Management System (PMS)

Submitted by: **Eyob Tsegaw**

Internship Hosting Organization: **IoT Center, Debre Tabor University**

Duration of Internship: July 1 – August 30

University Advisor: Mr. Abebe Kinde

Center Mentor and Supervisor: Mr. Abebe Kinde, Leader of the IoT Center

Date of Submission: October 2026

Debre Tabor, Ethiopia

Declaration, Approval of Mentor and Supervisor

Declaraition of the Student

I, Eyob Tsegaw, declare that this internship report, entitled “Development of a Pharmacy Management System (PMS)”, is the result of my own work. It describes what I did during my internship at the Internet of Things (IoT) Center of Debre Tabor University from July 1 to August 30. Every source of information that I used has been acknowledged in the text or in the list of references. The report has not been submitted to any other institution for a degree, diploma or any other qualification.

Name: Eyob Tsegaw Signature: _____ Date: _____

Approval of the Center Mentor and Supervisor

I confirm that the internship activities described in this report were carried out by the above-named student under my guidance and supervision at the IoT Center of Debre Tabor University, and I approve the report for submission.

Name: Mr. Abebe Kinde, Leader of the IoT Center Signature: _____ Date: _____

Center stamp:

Approval of the University Mentor / Advisor

This internship report is submitted for evaluation with my approval as the university advisor.

Name: Mr. Abebe Kinde Signature: _____ Date: _____

Acknowledgements

First of all, I thank Almighty God, who gave me the health, patience and strength to finish the internship and to write this report.

My sincere thanks go to the IoT Center of Debre Tabor University for accepting me as an intern and for giving me a real project to work on. I am most grateful to the Leader of the Center, Mr. Abebe Kinde, who acted as my university advisor, my Center mentor and my supervisor. His clear direction, his patience with my questions and his honest feedback helped me to turn classroom knowledge into a working system. He also arranged my visit to the MESOB installation site in Debre Tabor City Administration, where I could see a real network and security installation at close range.

I also thank the engineers and technicians at the MESOB site, who allowed me to watch their work and answered my questions about the equipment, and the pharmacy staff who explained their daily routines to me during the requirement-gathering stage.

I am thankful to Debre Tabor University, and in particular to the Research, Community Service, Technology Transfer and Industry Linkage Coordinator's office, for organizing the internship program, and to the instructors of the Department of Computer Science for the knowledge that prepared me for it. Finally, I thank my family and friends for their constant encouragement and moral support.

Executive Summary

This report describes my internship at the Internet of Things (IoT) Center of Debre Tabor University, carried out from July 1 to August 30 as part of the Bachelor of Science program in Computer Science. Most of my time went to software development, and a smaller part went to computer networking through a visit to the MESOB installation site of Debre Tabor City Administration.

The IoT Center is a technology and innovation unit of the university. It is located at the main gate of the campus and is led by Mr. Abebe Kinde. Its work covers Internet of Things prototyping, software development, support for network and ICT infrastructure, and practical guidance for students and interns. Chapter Two describes it in more detail.

My main assignment was to design and build a **Pharmacy Management System (PMS)**: a full-stack web application that moves the everyday work of a pharmacy from paper registers and spreadsheets into one secure system. It covers medicine inventory with low-stock and expiry alerts, a cart-based point of sale, prescription review, supplier and expense records, a real-time chat between patients and pharmacists, a public medicine catalog, customer feedback with e-mail replies, sales and profit dashboards, and an audit log of user actions.

The problem behind the project is that manual pharmacy records often cause stock mismatches, expired medicines that nobody notices, mistakes in sales records, weak control of prescriptions and slow reporting. PMS answers these problems with a role-based design for Admins and Pharmacists.

The front end uses **React 19** and **Tailwind CSS**, the back end uses **Node.js with Express 5**, the data are kept in **MySQL**, and **Socket.IO** provides real-time chat. Passwords are hashed with bcrypt and sessions use JSON Web Tokens (JWT). Customers can pay by cash, bank transfer or Telebirr (confirmed manually by the pharmacist).

During the networking part I visited the MESOB initiative, a government program that brings the social services of Debre Tabor City Administration together in one digital service center. I observed how servers, network switches, structured cabling and IP security cameras are installed, connected and tested as one system, and I linked this to what I had studied in my networking courses.

The main results are a working PMS prototype and a clearer understanding of how real network infrastructure is built. The main challenges were keeping stock consistent during

sales, handling different payment methods, evolving the database while the system was in use, and making the chat reliable. They were solved with database transactions, automatic migrations, Socket.IO rooms with stored history and role-based middleware. The internship improved my practical programming, problem solving, teamwork, communication and professional ethics. The report ends with conclusions and recommendations for the IoT Center, the university and the future development of the system.

List of Tables and Figures

List of Tables

- Table 1.1 – Summary of manual-system problems and proposed solutions
- Table 2.1 – Key facts about the host organization
- Table 2.2 – Summary of the main areas of activity of the IoT Center
- Table 2.3 – Main roles in the IoT Center
- Table 3.1 – Summary of work tasks and schedule
- Table 3.2 – Functional requirements
- Table 3.3 – System roles and access rights
- Table 3.4 – Main database entities
- Table 3.5 – Technology stack
- Table 3.6 – Network components observed during the visit
- Table 3.7 – Challenges and measures taken
- Table 3.8 – Main API endpoints
- Table 3.9 – Test cases
- Table 3.10 – Comparison of existing and proposed system
- Table 4.1 – Summary of skills gained

List of Figures

- Figure 2.1 – Organizational structure of the IoT Center
- Figure 2.2 – Work flow of project work at the IoT Center
- Figure 3.1 – System architecture of PMS
- Figure 3.2 – Use case diagram
- Figure 3.3 – Entity relationship diagram
- Figure 3.4 – Simplified topology of a network and camera installation like the one observed at MESOB
- Figure 3.5 – Server being configured during installation (own photo)
- Figure 3.6 – Monitoring screen with several camera views (own photo)
- Figure 3.7 – Reception area of the Debre Tabor MESOB service center
- Figure 3.8 – Self-service ticket kiosk with a touchscreen
- Figure 3.9 – Login page and Admin dashboard
- Figure 3.10 – Point-of-sale interface
- Figure 3.11 – Medicine inventory page

- Figure 3.12 – Real-time patient chat
- Figure 3.13 – Public product catalog

Table of Contents

Declaration, Approval of Mentor and Supervisor i
Acknowledgements ii
Executive Summary iii
List of Tables and Figures v
Abbreviations and Acronyms ix
Chapter One: Project Selection and Methodology 1
1.1 Project Title and Short Summary 1
1.2 Problem Statement and Justification 2
1.2.1 Problem Statement 2
1.2.2 Justification 2
1.3 Objectives of the Project 3
1.3.1 General Objective 3
1.3.2 Specific Objectives 4
1.4 Methodology 4
1.4.1 Development Approach 4
1.4.2 Data and Requirement Collection 4
1.4.3 System Analysis and Design Tools 5
1.4.4 Implementation Tools 5
1.4.5 Testing Method 5
1.5 Literature Review 5
1.5.1 Pharmacy Management Systems 5
1.5.2 Inventory and Expiry Control 6
1.5.3 Web-Based Architectures and REST APIs 6
1.5.4 Authentication and Role-Based Access Control 6
1.5.5 Real-Time Communication 6
1.5.6 Digital Payments in Ethiopia 6
1.5.7 Research Gap 7
Chapter Two: Background of the Internship Hosting Organization 8
2.1 Brief History 8
2.2 Main Products and Services 10
2.3 Main Customers and End Users 11
2.4 Organizational Structure 11
2.5 Work Flow 13

Chapter Three: Overall Internship Experience and Specific Work 15
3.1 Why I Selected the IoT Center 15
3.2 Section of the Organization Where I Worked and Why 15
3.3 Work Flow in the Section 16
3.4 Work Pieces and Tasks Executed 16
3.4.1 Overview of the Tasks 16
3.4.2 Requirement Analysis 17
3.4.3 System Design 19
3.4.4 Database Design 22
3.4.5 Implementation of the Modules 24
3.5 Methods, Tools and Techniques Used 25
3.6 Networking Exposure: The MESOB Project 27
3.6.1 Overview of the Project 27
3.6.2 Basic Network Concepts Needed to Understand the Installation 27
3.6.3 Network Components Observed 28
3.6.4 How the Components Work Together 28
3.6.5 Installation Process Observed 29
3.6.6 Photographs from the Visit 30
3.6.7 Monitoring of the Security Cameras 33
3.6.8 Good Practice Observed on Site 33
3.6.9 Link to My Academic Knowledge and to the PMS 33
3.7 Major Challenges and Problems Faced 34
3.8 Measures Taken to Overcome the Challenges 35
3.9 Results and Discussion 36
3.9.1 Results Achieved 36
3.9.2 System Testing 40
3.9.3 Discussion 42
3.9.4 Results of the Networking Exposure 43
3.10 Recommendations Regarding the Identified Problems 43
Chapter Four: Overall Benefits Gained from the Internship 45
4.1 Improvement of Practical Skills 45
4.2 Upgrading of Theoretical Knowledge 45
4.3 Improvement of Industrial Problem-Solving Capability 45
4.4 Improvement of Team-Playing Skills 46
4.5 Improvement of Leadership Skills 46
4.6 Work Ethics, Industrial Psychology and Related Issues 46
4.7 Entrepreneurship Skills 47

4.8 Interpersonal Communication Skills 47
Chapter Five: Conclusion and Recommendations 49
5.1 Conclusion 49
5.2 Recommendations for the IoT Center 49
5.3 Recommendations for the University 50
5.4 Future Work on the System 50
References 51
Appendices 51
Appendix A – Installation and Running Guide 51
Appendix B – Environment Variables 52
Appendix C – Project Folder Structure 52
Appendix D – Interview / Observation Questions 53
Appendix E – Additional Screenshots and Weekly Activity Log 54

Abbreviations and Acronyms

Abbreviation	Meaning
API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTU	Debre Tabor University
ERD	Entity Relationship Diagram
ETB	Ethiopian Birr
FEFO	First Expired, First Out
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
IP	Internet Protocol
JWT	JSON Web Token
LAN	Local Area Network
MESOB	Government one-stop digital service initiative (Debre Tabor City Administration site)
PMS	Pharmacy Management System
PoE	Power over Ethernet
POS	Point of Sale
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language
UI	User Interface
UPS	Uninterruptible Power Supply
UTP	Unshielded Twisted Pair

Chapter One: Project Selection and Methodology

1.1 Project Title and Short Summary

Project title: Development of a Pharmacy Management System (PMS)

The Pharmacy Management System is a full-stack web application that streamlines daily pharmacy operations through role-based dashboards for **Admins** and **Pharmacist**. It manages medicine inventory, real-time sales, prescription upload, supplier records, expenses and financial reporting. In addition, it offers a patient live-chat consultation channel and a public-facing product catalog, so that customers can see which medicines are available and communicate with the pharmacy without visiting in person.

The main features of the system are:

- Secure JWT-based login with role-based access control and a complete audit log of user actions.
- Inventory management: adding, editing and deleting medicines with image upload, categorization (tablet, capsule, syrup, injection and others), low-stock alerts, expiry alerts, stock inward logging and wastage tracking (expired, damaged, lost).
- A cart-based Point of Sale with real-time stock validation and automatic stock decrement after a sale is confirmed.
- Prescription management: upload of prescription images, and approval or rejection by pharmacists with status tracking (pending, approved, rejected).
- Payment processing by cash, bank and telebirr, with the payment method stored for each sale.
- Real-time patient chat built on Socket.IO with notifications, unread badges and stored chat history.
- A public product catalog that reads live from the backend and shows an “Out of Stock” status based on database quantity.
- Customer feedback with an admin inbox and email replies sent through Nodemailer.
- Admin dashboard with total, daily, weekly, monthly and yearly sales, total expenses and profit, plus a cashier daily report.

The project was carried out as a practical internship assignment at the IoT Center of Debre Tabor University, under the supervision of Mr. Abebe Kinde, and its source code is organized into a `backend` directory (Express API, controllers, models, routes, middleware,

SQL schema, file uploads) and a `frontend` directory (React components, pages, context and services).

1.2 Problem Statement and Justification

1.2.1 Problem Statement

A pharmacy handles a large number of items with different batches, prices and expiry dates. It also handles prescriptions that must be verified, customers who pay in different ways, and suppliers who must be paid and tracked. In many pharmacies, these activities are still performed manually using paper registers, receipt books or basic spreadsheets. The following problems are commonly observed in such a manual environment:

1. **Inaccurate stock records.** Quantities written in registers do not always match the physical shelf, so the pharmacy either runs out of fast-moving medicines or over-orders slow-moving ones.
2. **Expired medicines remain unnoticed.** Without automatic expiry tracking, expired medicines may stay on the shelf and may even be sold, which is a serious patient-safety and legal risk.
3. **Errors in sales recording.** Manual calculation of totals and manual deduction from stock lead to arithmetic errors, missing entries and lost receipts.
4. **Weak prescription control.** Medicines that require a prescription may be dispensed without proper verification, and prescription records are difficult to retrieve later.
5. **Slow and unreliable reporting.** Daily, weekly or monthly sales, expenses and profit must be calculated by hand, which is time consuming and error prone.
6. **No accountability.** When stock or cash is missing, it is hard to know who did what and when, because there is no audit trail.
7. **Poor customer communication.** Patients have no easy way to check whether a medicine is available or to consult a pharmacist before travelling to the pharmacy.
8. **Limited payment recording.** Payments through bank transfer or mobile money are not recorded consistently, which complicates daily cash reconciliation.

1.2.2 Justification

A computerized pharmacy management system is justified for the following reasons:

- **Accuracy:** Automatic calculation and automatic stock decrement within database transactions remove most manual errors.

- **Patient safety:** Expiry alerts, prescription-required detection and pharmacist approval reduce the chance of dispensing expired or controlled medicines incorrectly.
- **Efficiency:** Pharmacists spend less time on paperwork and more time on patient care.
- **Transparency and control:** Role-based access and audit logs make every important action traceable.
- **Better decisions:** Real-time dashboards give the owner immediate insight into sales, expenses and profit.
- **Customer service:** The public catalog and live chat make the pharmacy more accessible, in line with the growing use of mobile and digital payments in Ethiopia (for example Telebirr and Bank).

Table 1.1: Summary of manual-system problems and proposed solutions

Problem in the manual system	Proposed solution in PMS
Stock mismatch	Real-time stock validation and automatic decrement on sale
Unnoticed expiry	Expiry-date monitoring with alerts
Slow reports	Dashboard KPIs computed by MySQL queries
No accountability	Audit logs for every user action
Prescription misuse	Prescription upload, review, approve or reject flow
Payment confusion	Payment method stored per sale and shown in daily report
Poor customer contact	Public catalog and Socket.IO live chat

1.3 Objectives of the Project

1.3.1 General Objective

To design and develop a secure, role-based and real-time web-based Pharmacy Management System that automates the main operations of a pharmacy and improves accuracy, efficiency and customer service.

1.3.2 Specific Objectives

- To study and analyze the existing manual pharmacy operations and identify user requirements.
- To design the system architecture, database and user interfaces for the Admin and Pharmacist roles.
- To implement authentication, authorization and audit logging using JWT and bcrypt.
- To implement medicine inventory management with image upload, categories, stock-inward, wastage tracking, low-stock and expiry alerts.
- To implement a point-of-sale module that validates stock, records the payment method and updates stock automatically.
- To implement prescription upload and review with status tracking.
- To integrate manual payment options, telebirr and bank.
- To implement a real-time chat between patients and pharmacists, and a customer feedback inbox with email reply.
- To implement dashboards and reports for sales, expenses and profit.
- To test the system and evaluate whether it meets the stated requirements.

1.4 Methodology

1.4.1 Development Approach

The project followed an **iterative and incremental** software development approach. Instead of building the whole system in one pass, I delivered it module by module: authentication first, then inventory, sales, payments, prescriptions, chat, feedback and reports. After each iteration the module was reviewed with the supervisor, tested, and improved. The backend performs an automatic database migration at startup, which made it easy to add new columns in later iterations without rebuilding the database.

1.4.2 Data and Requirement Collection

Requirements were gathered through:

- **Observation** of how pharmacy work is carried out: receiving goods, shelving, selling, handling prescriptions and daily cash counting. State which pharmacy or pharmacies you visited.
- **Informal interviews and discussions** with pharmacists, and with my supervisor Mr. Abebe Kinde about their daily problems and wishes.

- **Document review** of the existing registers, receipts and reports used in the manual process.

1.4.3 System Analysis and Design Tools

- Use case diagrams to identify actors (Admin, Pharmacist, Patient/Customer) and their interactions.
- Entity Relationship Diagrams and a SQL schema (`schema.sql`) to model the database.
- UI mock-ups and component-based design in React.
- A layered backend structure: routes, controllers, models, middleware and configuration.

1.4.4 Implementation Tools

The implementation used React 19, Vite, React Router v7, Tailwind CSS v4, Material UI, Axios, Recharts and Socket.IO Client on the frontend; Node.js, Express 5, MySQL2, Socket.IO, jsonwebtoken, bcryptjs, Multer and Nodemailer on the backend; and MySQL as the database. Details are given in Chapter Three.

1.4.5 Testing Method

The system was tested through functional (black-box) testing of every module, integration testing between the React frontend and the Express API, and demonstration and feedback sessions with my supervisor.

1.5 Literature Review

1.5.1 Pharmacy Management Systems

A pharmacy management system is a software solution that supports the management of medicines, sales, prescriptions, patients, suppliers and finances in a pharmacy. Studies and commercial products in this area generally agree on a core set of functions: inventory control with reorder and expiry alerts, billing and point of sale, prescription handling, supplier management and reporting. Adopting such systems is widely reported to reduce medication and stock errors, save staff time and improve profitability, although the benefit depends on how well the system matches local workflow.

1.5.2 Inventory and Expiry Control

Medicine inventory differs from ordinary retail inventory because every batch has an expiry date and incorrect storage or sale can harm patients. Good practice therefore requires batch-level records, minimum-stock thresholds, first-expired-first-out (FEFO) handling and a record of wastage with its reason (expired, damaged or lost). PMS reflects this practice by providing stock-inward logging, low-stock alerts, expiry alerts and a wastage log.

1.5.3 Web-Based Architectures and REST APIs

Modern business applications are usually built as single-page applications that communicate with a server through REST APIs. A React frontend renders the interface in the browser and exchanges JSON with an Express server, which applies business rules and talks to the database. This separation makes the system easier to maintain, lets the frontend and backend evolve independently, and allows the same API to be reused later by a mobile application.

1.5.4 Authentication and Role-Based Access Control

JSON Web Tokens allow a server to authenticate requests without keeping a server-side session: after login, the client sends a signed token with each request, and middleware verifies it. Combined with role-based access control (RBAC), this restricts each endpoint to the roles that need it. Passwords must never be stored in plain text; hashing with bcrypt, which is deliberately slow and salted, is the accepted practice and is the approach used in PMS.

1.5.5 Real-Time Communication

HTTP is a request-response protocol and is not ideal for instant messaging. WebSocket-based libraries such as Socket.IO keep a persistent connection between browser and server so that messages are delivered immediately and the server can push notifications. This is the basis of the patient-to-pharmacist chat in PMS, where chat history is also stored in the database so that conversations survive page reloads.

1.5.6 Digital Payments in Ethiopia

Mobile money and online payment services such as Telebirr and bank. A pharmacy system that can record these payment methods, and integrate a gateway for online checkout, matches current customer behavior and simplifies daily reconciliation.

1.5.7 Research Gap

Many available systems are either expensive foreign products that do not support local payment methods, or simple desktop programs without real-time communication and public catalogs. The project therefore combines inventory, POS, prescription control, Ethiopian payment options, patient chat and a public catalog in one web-based, affordable and open technology stack.

Chapter Two: Background of the Internship Hosting Organization

2.1 Brief History

My internship took place at the **Internet of Things (IoT) Center of Debre Tabor University** (called “the IoT Center” or “the Center” in this report). The Center belongs to my own university, so I first describe the university and then the Center.

Debre Tabor University. Debre Tabor University (DTU) is a public university in the historic town of Debre Tabor, the capital of the South Gondar Zone in the Amhara Region. The town sits at the foot of the Guna mountain range. The campus was founded close to Gafat, the place linked to the first industrial workshops built under Emperor Tewodros II, and this history is the reason why the university’s motto speaks of a “historical responsibility” to answer Tewodros’s quest for knowledge.

The university was established in 2008 G.C. (2001 E.C.). It started with 628 students at the former Wereta Agricultural College and has since grown to more than 18,000 students in undergraduate, postgraduate and doctoral programs. It has several colleges, a technology institute and a school of law, and it runs about 75 regular and extension programs together with a number of research centers that work with farmers and local communities. My department, Computer Science, belongs to the Gafat Institute of Technology.

The IoT Center. The IoT Center is a technology and innovation unit of the university. It is placed at the main gate of the campus, so students, staff and visitors can reach it easily, and it is led by **Mr. Abebe Kinde**, who was also my advisor and supervisor. The Center was established in 2001 to support practical learning and innovation, mainly in the Internet of Things.

The Internet of Things means connecting physical objects such as sensors, cameras and machines to a network so that they can send data and be monitored or controlled from a distance. The Center uses this idea as the base for project-based learning in which students combine software, electronics and networking to solve practical problems.

The Center is still growing. Its equipment is limited and its organization is simple. I mention this openly because it was part of my real working environment, and it explains some of the recommendations in Chapter Five. On the positive side, the leader takes part in practical

work himself, including the MESOB installation, and the location in the university makes it a natural place where students can gain experience.

Table 2.1: Key facts about the host organization

Item	Description
Name	Internet of Things (IoT) Center, Debre Tabor University
Location	Main gate of the Debre Tabor University campus, Debre Tabor, Ethiopia
Leader	Mr. Abebe Kinde
Type of organization	Public university technology and innovation center
Main fields	IoT, software development, network and ICT support, student guidance

Vision and mission of the university. The vision of Debre Tabor University is to become the leading comprehensive university in Ethiopia by 2030 G.C. Its mission is to contribute to national development by producing competent graduates, by excelling in agriculture and environmental science, health sciences and technology, and by solving the problems of society through research that is combined with indigenous knowledge. The Center follows the technology part of this mission by turning knowledge into working solutions and by giving students practical experience.

Vision and mission of the Center. The Center had no written vision or mission available to me during the internship. If it has one, copy it here. From what I observed, its purpose can be summarized as follows:

- to promote practical learning and innovation in IoT and related technologies;
- to build software and IoT-based solutions for real needs of the university and the community;
- to support network and ICT infrastructure work;
- to give students and interns real project experience; and
- to cooperate with the university, government offices and other partners on technology projects.

2.2 Main Products and Services

The Center works in the following areas. Because it is small and has limited equipment, some of them are carried out on a modest scale.

IoT projects and prototyping. Sensors, controllers and network connections are combined with software to monitor or control something. Prototypes are built and tested before real use. The number of boards, sensors and devices available limits how much can be done at the same time.

1. **Software development.** The Center supports the development of web-based and other software systems. The Pharmacy Management System that I built during the internship is an example of this kind of work.
2. **Network and ICT infrastructure.** The Center Leader takes part in the installation and configuration of network and security equipment. This gave me the chance to observe a real installation, described below and in Section 3.6.
3. **Student guidance and internships.** Students and interns receive real tasks, regular feedback and direct mentoring from the leader.

The MESOB initiative (external exposure). MESOB is a national government initiative. In Debre Tabor, its goal is to bring the social services of the City Administration together in one place and to deliver them digitally, so that citizens are served faster and with less travel. MESOB is not a project of the university or of the Center, so I do not count it among the Center's own services. However, Mr. Abebe Kinde works on its installation and configuration phase, and he arranged my visit to the site.

Table 2.2: Summary of the main areas of activity of the IoT Center

Area	Description	Main beneficiaries
IoT projects and prototyping	Building and testing IoT-based solutions, within the limits of available equipment	University, students
Software development	Web and other software systems, such as the PMS developed during this internship	University and other institutions

Area	Description	Main beneficiaries
Network and ICT infrastructure	Participation of the Center Leader in installation and configuration work	Public institutions, the university
Student guidance	Practical experience and mentoring for students and interns	Students

2.3 Main Customers and End Users

Since the Center is part of a public university, the people it serves are partners and beneficiaries more than paying customers.

- **Students and interns,** who use the Center for learning, projects and internships.
- **Academic and administrative staff,** who can use its solutions and technical support in teaching, research and administration.
- **Debre Tabor University** as an institution, which gains systems, skills and innovation from the Center's work.
- **Debre Tabor City Administration and other government offices,** which are partners of the university; the City Administration is also the beneficiary of the MESOB initiative.
- **Local institutions, businesses and the community,** such as pharmacies, which can benefit from locally built software.

The **end users** are the people who use a delivered system every day. For the PMS, these would be the Admin, the pharmacists and the cashiers of a pharmacy, and the patients who use its public catalog and chat. For infrastructure projects such as MESOB, they are the staff who operate the network, servers and cameras, and the citizens who finally receive the services. Knowing who the end users are is important, because a system is only successful when these people find it easy to use and can trust it.

2.4 Organizational Structure

The Center is led by **Mr. Abebe Kinde**, who sets its direction, plans and supervises its work, and keeps contact with the university management and with outside partners.

During my internship it did not have formally defined departments. Work was arranged around projects and tasks that the leader assigned directly to developers, students and interns, as shown in Figure 2.1.

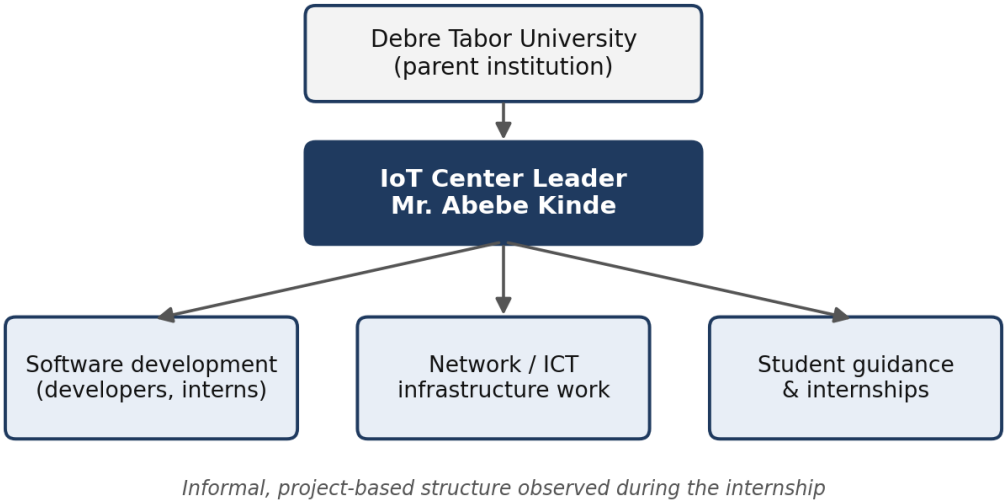


Figure 2.1 – Organizational structure of the IoT Center as observed during the internship

Table 2.3: Main roles in the IoT Center

Role	Main responsibilities
Center Leader (Mr. Abebe Kinde)	Sets direction, plans and supervises projects, coordinates with the university and partners, works on infrastructure installation, and mentors interns
Mentor, supervisor and university advisor	During my internship, all three roles were carried out by the Center Leader
Developers, students and interns	Carry out assigned tasks such as software modules and report progress to the leader

I worked mainly on the software project and received guidance, feedback and approval directly from Mr. Abebe Kinde. For the networking part, I visited the MESOB installation site, with the visit arranged by him.

A small structure with direct contact between leader and team has clear advantages, because decisions are quick and guidance is personal. Its weakness is that the work depends heavily on one person, and that it is harder to share tasks, follow progress and keep documentation when roles and procedures are not written down. Strengthening the structure is one of my recommendations in Chapter Five.

2.5 Work Flow

The work flow in the Center is practical and informal rather than a fixed written procedure. Figure 2.2 shows the sequence that I experienced, and the steps are explained below.

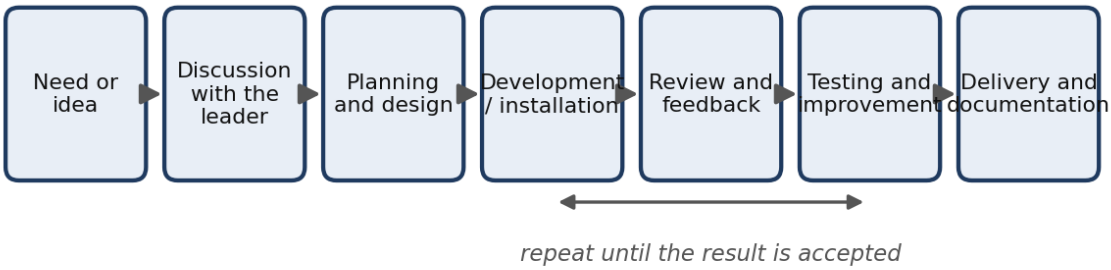


Figure 2.2 – Work flow of project work at the IoT Center as experienced during the internship

1. **Need or idea.** A task begins with a need of the university, a partner or the Center itself, or with a new idea.
2. **Discussion with the leader.** Mr. Abebe Kinde explains the goal and the expected result.
3. **Planning and design.** The scope, main functions and approach are agreed.
4. **Development or installation.** Software is written, or equipment is mounted and configured.
5. **Review and feedback.** Progress is shown to the leader, who comments on it.
6. **Testing and improvement.** Problems are corrected and the result is tested again.
7. **Delivery and documentation.** The result is handed over and documented as far as time and resources allow.

For software work, such as the PMS, I tested each feature on my own computer and showed it to Mr. Abebe Kinde for review before starting the next one The installation work that I observed at MESOB followed the usual stages of a network installation: survey and planning, cable routing, mounting of equipment, connection, configuration, testing and handover. Section 3.6 describes these stages.

Facilities and working environment. The Center is easy to reach, since it stands at the campus gate, and its leader was also my advisor, which made guidance direct. Equipment and workspace are limited, so I had to plan my own tasks, keep my own notes and find solutions on my own. together with free and open-source tools (Visual Studio Code, Node.js,

MySQL and Git). This showed me that useful software can be built with modest resources, but also that more computers for testing, IoT devices and network equipment for practice would let the Center support more students.

Chapter Three: Overall Internship Experience and Specific Work

3.1 Why I Selected the IoT Center

I chose the IoT Center of Debre Tabor University for these reasons:

- **It is part of my own university and matches my field.** The Center works with technology and innovation, which is close to what I study in Computer Science.
- **It offers real software work.** I was able to take a complete system from requirement analysis to a working prototype, which fits my goal of becoming a full-stack developer.
- **It adds a networking side.** The Center Leader is involved in real network and security installations, so I could build software and, in the same internship, see how a physical network is installed.
- **It gave me modern tools.** I could work with a current stack (React 19, Express 5, MySQL and Socket.IO) and integrate services such as Chapa and Nodemailer.
- **It offers direct mentorship.** The leader was also my advisor, so I received close guidance and quick feedback.
- **It is convenient.** The Center is at the main gate of the campus, so it was easy to reach every day.

I picked a pharmacy system as my project because errors in stock, expiry dates and prescriptions directly affect patient safety, so software can make a visible difference in that field.

3.2 Section of the Organization Where I Worked and Why

I worked in the **software development** side of the Center under the guidance of Mr. Abebe Kinde. This was a good fit because my interest is application development, and the PMS gave me a project that I could follow from the first idea to a working prototype. To understand the people who would use the system, I also observed how pharmacy work is done: the counter, the store and the cashier desk.

For the networking part, I visited the installation site of the **MESOB** initiative of Debre Tabor City Administration. The visit was arranged by Mr. Abebe Kinde, who is involved in the installation and configuration phase of the project. MESOB is run by the government and not by the university.

3.3 Work Flow in the Section

In the software section the work followed these steps:

1. Mr. Abebe Kinde and I identified a need or problem in daily pharmacy operations.
2. I analyzed the need, asked questions about the parts that were unclear and wrote the requirement.
3. I designed the database change, the API endpoint and the user interface.
4. I implemented the feature and tested it on my development computer.
5. The supervisor reviewed the feature and I corrected the problems he found.
6. I moved to the next feature in the same way.

Work was planned weekly, with a short review at the end of each week to show what had been completed and to agree on the next tasks.

3.4 Work Pieces and Tasks Executed

The main task of the internship was the design and development of the Pharmacy Management System. The work was divided into the following pieces.

3.4.1 Overview of the Tasks

Table 3.1: Summary of work tasks and schedule

Period	Main activities	Output
Weeks 1–2	Orientation at the Center, observation of pharmacy work, requirement gathering and analysis	Requirement list
Weeks 3–4	System design and database design; authentication module	Design, schema, login and roles
Weeks 5–7	Medicine inventory, point of sale and payment methods (including Chapa)	Working inventory, POS and payments
Weeks 8–9	Prescription module, real-time patient chat, public catalog, feedback with e-mail reply	Prescription review, chat, catalog, inbox
Weeks 10–11	Dashboards, reports, audit logs, testing and bug fixing	Reports and test results

Period	Main activities	Output
Week 12	Documentation, user demonstration and final report	Final report
	Visit to the MESOB network installation site, arranged by the Center Leader	Understanding of network infrastructure

The software tasks were: studying pharmacy operations and listing roles and functions; designing the architecture, database, API and interface; building the Express API with MySQL; building the React front end; implementing JWT login, bcrypt hashing and role-based access; building inventory, POS, prescription, chat, catalog, feedback and report modules; testing and fixing defects; and writing documentation.

The networking tasks were: visiting the MESOB site, observing how servers, switches, cabling and IP cameras are installed and configured, asking the engineers and technicians about the purpose and connection of each device, and learning how devices are powered, labeled and tested.

3.4.2 Requirement Analysis

Table 3.2: Functional requirements

ID	Requirement	Actor
FR-01	Register and log in users with role-based access	All staff
FR-02	Add, edit, delete and search medicines with images and categories	Pharmacist, Admin
FR-03	Record goods received (stock inward)	Pharmacist
FR-04	Show low-stock and near-expiry medicines	Pharmacist, Admin
FR-05	Record wastage (expired, damaged, lost)	Pharmacist

ID	Requirement	Actor
FR-06	Sell medicines through a cart-based POS with stock validation	Pharmacist
FR-07	Record payment method cash, bank, Telebirr for each sale	Pharmacist,
FR-08	Upload prescriptions	Pharmacist
FR-09	Real-time chat between patients and pharmacists	Patient, Pharmacist
FR-10	Public catalog of medicines with availability status	Customer
FR-11	Submit feedback and reply by email	Customer, Admin
FR-12	Manage users, suppliers and expenses	Admin
FR-13	Show dashboards and daily reports	pharmacist
FR-14	Record and display audit logs	Admin

Non-functional requirements:

- **Security:** passwords are hashed with bcrypt; access is controlled by JWT and role checks; secrets are kept in environment variables and never committed to the repository.
- **Performance:** dashboard totals are computed by MySQL on the server so that the browser receives only the result.
- **Reliability and data integrity:** sales are saved inside database transactions so that the sale record and the stock update succeed or fail together.
- **Usability:** a clean responsive interface, color-coded badges, toast notifications and quick-reply templates.
- **Maintainability:** separate routes, controllers, models and middleware on the backend, and a component-based structure on the frontend.

- **Portability:** the system runs on Node.js v18+ and MySQL v8+ on any common operating system.

Table 3.3: System roles and access rights

Role	Access
Admin	User management, supplier management, expense tracking, audit logs, feedback inbox, full dashboard
Pharmacist	Medicine inventory and image upload, stock management, POS, prescription uploads, wastage log, patient chat, payment processing, daily reports and orders
Patient / Customer (public)	Product catalog, chat page, contact form

3.4.3 System Design

The system follows a three-tier architecture: a React single-page application (presentation tier), an Express REST API with a Socket.IO server (application tier), and a MySQL database (data tier). Static files such as medicine images and prescription images are stored on the server in the `uploads` directory and served through the `/uploads` path.

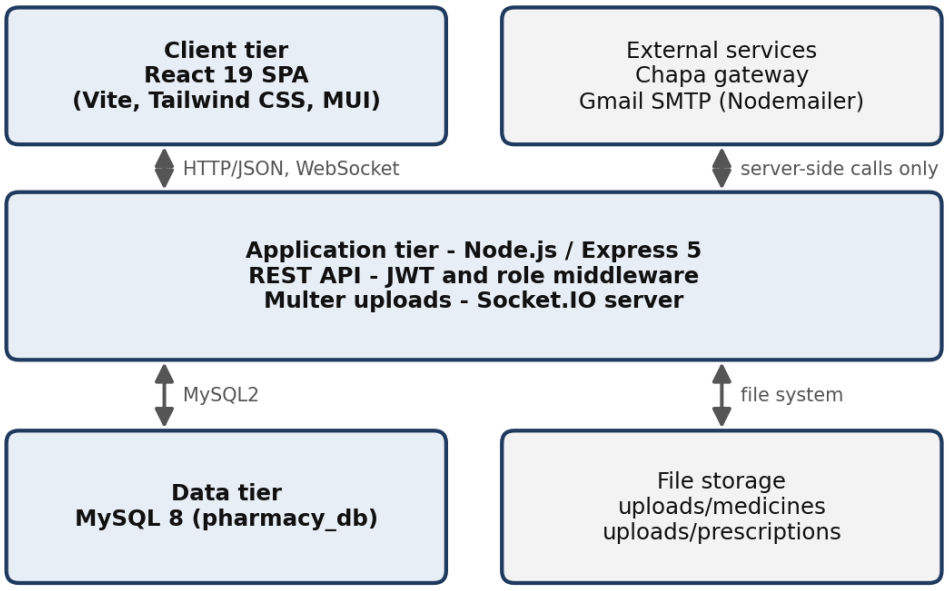


Figure 3.1 – System architecture of PMS

The tiers are summarized below.

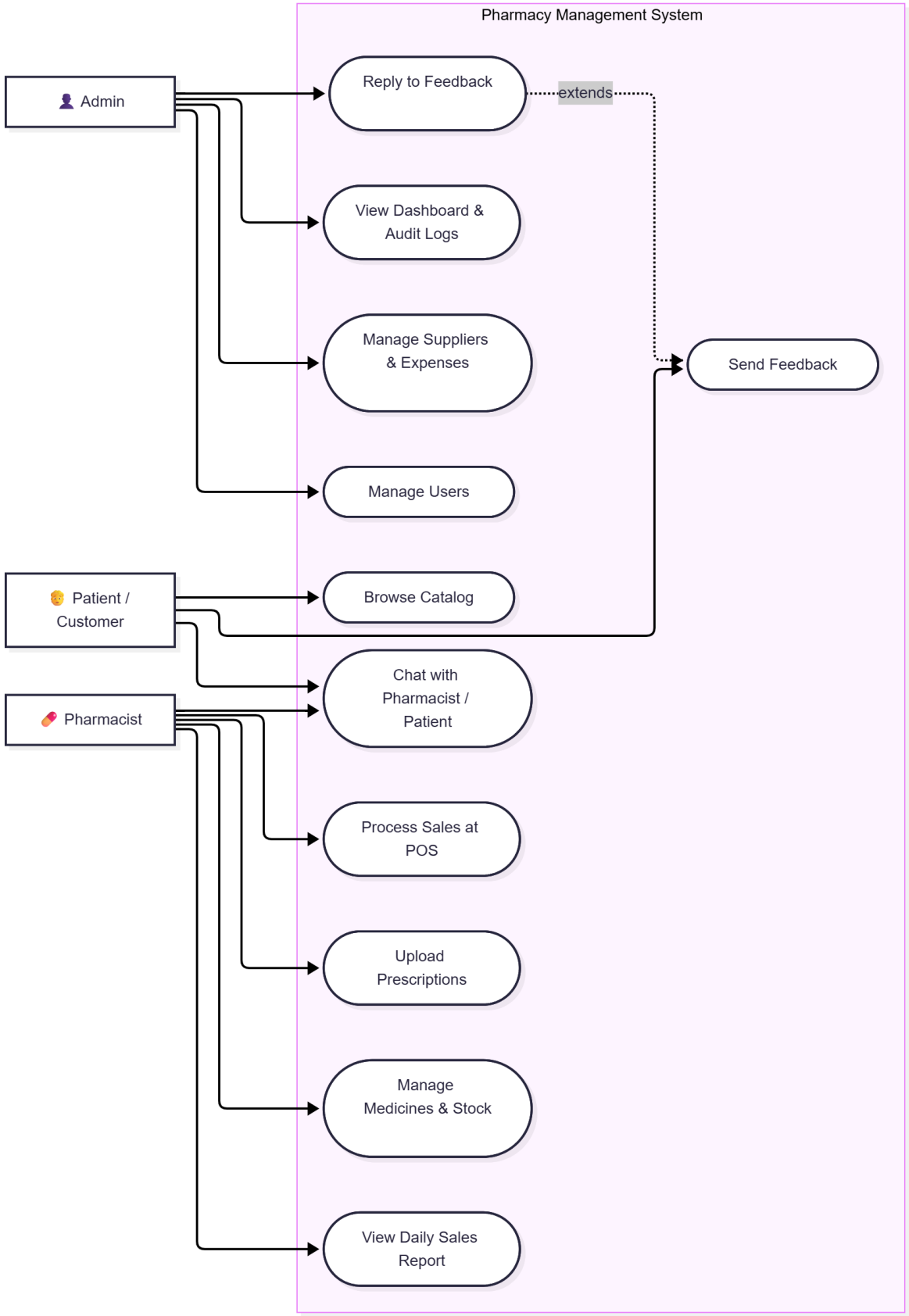
Tier	Components	Communication
Client	React 19 single-page app (Vite, Tailwind CSS, Material UI)	HTTP/JSON through Axios; WebSocket through Socket.IO Client
Server	Express 5 REST API, Socket.IO server, JWT and role middleware, Multer uploads	Reads and writes MySQL and the uploads folder
Data	MySQL database; uploads folder for medicine and prescription images	Accessed through MySQL2 and the file system
External	Chapa payment gateway; Gmail SMTP through Nodemailer	Called by the server only

The backend is organized into:

- `config/database.js` – MySQL connection pool.
- `controllers/` – business logic for authentication, medicines, sales, suppliers, users, audit logs and prescriptions.
- `models/` – data access for Medicine, Sale, Supplier and User.
- `routes/` – route definitions for auth, medicines, sales, suppliers, users, categories, inventory, expenses, audit logs, prescriptions, prescription requests, chat and contact.
- `middleware/` – `auth.js` (protect and authorize) and `errorHandler.js`.
- `sql/schema.sql` – the complete database schema.
- `server.js` – the application entry point, which also starts Socket.IO and performs automatic migrations.

The frontend is organized into `components` (layout, sales, currency formatter, home, about, product and contact), `context` (AuthContext), `pages` (admin, pharmacist, cashier, chat, main and login) and `services` (Axios instance with base URL and authorization header).

Figure 3.2 – Use case diagram



The actors and their main use cases are listed below.

Actor	Main use cases
Admin	Manage users; manage suppliers and expenses; view dashboard and audit logs; reply to feedback
Pharmacist	Manage medicines and stock; process sales at POS; Upload prescriptions; chat with patients and view daily sales report
Patient / Customer	Browse catalog; chat with pharmacist; send feedback

3.4.4 Database Design

The database `pharmacy_db` is created from `backend/sql/schema.sql`. It is relational, and the sale process uses transactions. The main entities are listed below.

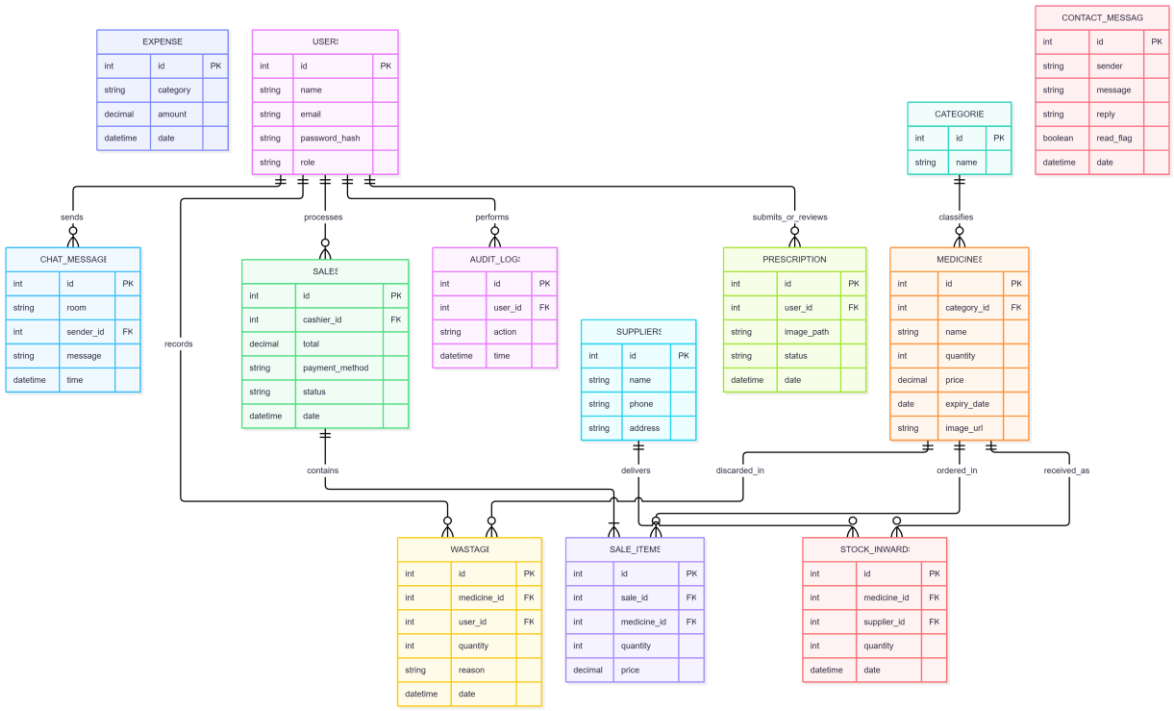
Table 3.4: Main database entities

Entity	Purpose	Important fields (examples)
users	System users and their roles	name, email, password hash, role
medicines	Medicine records	name, category, quantity, price, expiry date, `image_url`
categories	Medicine categories	name
suppliers	Supplier information	name, phone, address
sales	Sale header	cashier, total, `payment_method`, `status`, date
sale items	Medicines within a sale	sale, medicine, quantity, price
inventory / stock inward	Goods received log	medicine, supplier, quantity, date

Entity	Purpose	Important fields (examples)
wastage	Expired, damaged and lost items	medicine, quantity, reason
expenses	Pharmacy expenses	category, amount, date
prescriptions and prescription requests	Prescription images and review status	image path, `status` (pending, approved, rejected)
chat messages	Stored patient–pharmacist chat	room, sender, message, time
contact messages	Customer feedback and replies	sender, message, reply, read flag
audit logs	Who did what and when	user, action, time

diagram as Figure 3.3.

Figure 3.3 – Entity relationship diagram



3.4.5 Implementation of the Modules

(a) Authentication and authorization. Users register and log in through `/api/auth/register` and `/api/auth/login`. Passwords are hashed with bcryptjs (the number of rounds is set by `BCRYPT_ROUNDS`). After login the server returns a JWT that expires according to `JWT_EXPIRE`. The `protect` middleware verifies the token and the `authorize` middleware allows only the roles that are permitted on a route. On the frontend, `AuthContext` stores the user and token, and `ProtectedRoute` prevents unauthorized access to pages. Important actions are written to the audit log.

(b) Inventory management. Pharmacists can add and edit medicines through a form that supports image upload (Multer on the server, saved under `uploads/medicines`). The medicine list shows an image column and highlights low-stock items. Categories are managed in a category manager. The stock-inward page records received goods and increases the stock, while the wastage page records expired, damaged or lost items and reduces the stock. Special endpoints return low-stock and expired medicines for the alert pages and dashboard counts.

(c) Point of Sale. The POS is a cart-based interface. When a pharmacist adds a medicine, the system checks the available quantity. At checkout, the sale and its items are written to the database in one transaction, and the stock quantity is decremented automatically. If a medicine requires a prescription, the system detects this and starts the prescription upload flow before the sale can be completed. A receipt is printed using React To Print.

(d) Payment processing. Cash payments are recorded instantly. For bank and Telebirr payments, the pharmacist confirms how the customer paid and the named payment method is saved in the `payment_method` column of the `sales` table (a VARCHAR column that accepts any bank name). In the daily report, payment methods are shown as color-coded badges: Cash (green), Telebirr (blue) and Bank (amber).

(e) Prescription management. Prescription images are uploaded at checkout and stored in `uploads/prescriptions`. Pharmacists see a list of pending prescriptions and can approve or reject each one, and the status is tracked as `pending`, `approved` or `rejected`.

(f) Real-time patient chat. Patients open a public chat page and send messages to pharmacists through Socket.IO rooms. Pharmacists receive toast notifications with one-click navigation to the room. The sidebar shows color avatars with the patient's initials in a

Telegram-like style, and unread message badges per conversation. Messages are stored in MySQL and loaded when a room is joined, and pharmacists can use quick-reply templates to answer common questions faster.

(g) Public product catalog. The public page fetches the medicine list from the backend. It displays the uploaded images, shows an "Out of Stock" badge and disables the "Call to Order" button when the database quantity is zero, and offers a phone link for ordering.

(h) Customer feedback and email reply. Customers submit messages through the public contact form. The Admin sees them in a split-panel inbox with unread badges and a read/unread toggle. When the Admin replies, the reply is saved in the database and emailed to the customer through Nodemailer with Gmail SMTP (using an app password stored in `EMAIL_PASS`).

(i) Dashboards and reports. The Admin dashboard presents total sales, today's, weekly, monthly and yearly sales (calendar-based), total expenses and profit, all computed by MySQL queries on the backend through `/api/payments/stats`. A Recharts area chart shows the sales trend, and counters show low-stock and expiring medicines. The cashier's daily report lists sales filtered by cashier and date and shows the payment-method badges.

(j) Audit logs. Every important user action is recorded with the user and the time, and only the Admin can view this history.

3.5 Methods, Tools and Techniques Used

Since my field is software engineering, the methods, tools and techniques used in the work tasks were the following.

Table 3.5: Technology stack

Layer	Technology	Purpose
Frontend	React 19	User interface framework
Frontend	Vite 8	Build tool and development server
Frontend	React Router v7	Client-side routing
Frontend	Tailwind CSS v4	Styling

Layer	Technology	Purpose
Frontend	Material UI v9, Lucide React	Icons and UI components
Frontend	Axios	HTTP requests
Frontend	Socket.IO Client	Real-time chat
Frontend	Recharts	Charts and graphs
Frontend	React Hot Toast	Notifications
Frontend	React To Print	Receipt printing
Backend	Node.js with Express 5	Server and REST API
Backend	MySQL2	Database driver
Backend	Socket.IO	Real-time WebSocket server
Backend	jsonwebtoken	Authentication tokens
Backend	bcrypt	Password hashing
Backend	Multer	File uploads (medicines and prescriptions)
Backend	Nodemailer	Email replies to customers
Backend	dotenv, Nodemon	Environment configuration and development restart
Database	MySQL v8	Relational database with transactions

Techniques and practices applied:

- **Requirement analysis** through observation, interviews and document review.
- **Modeling** with use cases and entity relationship diagrams.
- **RESTful API design** with clear resource-based endpoints and HTTP methods.
- **Role-based access control** using middleware.
- **Database transactions** to keep sale and stock data consistent.
- **Automatic migration at startup** to evolve the database schema safely.
- **Component-based UI design** and shared state through React Context.

- **Environment-based configuration** to keep secrets out of source code.
- **Real-time event-driven communication** with Socket.IO rooms.
- **Testing and debugging** with browser developer tools, API testing tools and console logs.
- **Version control** with Git [and GitHub], to keep a history of changes.

3.6 Networking Exposure: The MESOB Project

Besides software development, I spent part of the internship learning about computer networking by visiting the **MESOB** installation site of Debre Tabor City Administration. My role was mainly to **observe and learn**: I watched the work, asked the engineers and technicians questions and compared what I saw with my networking courses.

3.6.1 Overview of the Project

MESOB is a national initiative of the Ethiopian government. In Debre Tabor it is meant to gather the social services of the City Administration under one roof and deliver them digitally, so that citizens do not need to move between many offices. This requires a dependable network that connects servers, office computers, self-service devices and security cameras. The phase I saw was the installation and configuration phase: cabling, mounting of equipment, connecting devices and configuring them so they work together.

3.6.2 Basic Network Concepts Needed to Understand the Installation

- **LAN (local area network):** a network inside one building or site that links computers, servers and cameras.
- **Switch:** a device that connects many devices in a LAN and sends data only to the port of the intended receiver.
- **Router and firewall:** a router links different networks, and a firewall decides which traffic may pass.
- **Server:** a computer that offers services to other devices, such as applications, storage or video recording.
- **IP camera:** a camera with its own network connection that sends digital video to a recording server.
- **PoE (Power over Ethernet):** power delivered to a device, such as a camera, through the same cable that carries its data
- **Structured cabling:** a standard way of arranging cables, patch panels and racks so the network is tidy, easy to maintain and ready to grow.

- **IP addressing:** every device on a network has an address by which it is identified and reached.

3.6.3 Network Components Observed

Table 3.6: Network components observed during the visit

Component	Role in the system
Servers	Run applications and services, store data and video recordings
Network switches	Connect local devices and pass data between them
IP security cameras	Capture video and send it over the network
Network cables	Carry data between devices
Patch panels and network rack	Hold the equipment and keep cable connections organized
Connectors and accessories (RJ45, cable trays, labels)	Complete and organize the physical connections
Power supply / UPS	Give the equipment stable power
Router, firewall, wireless access points	Connect and protect the network

3.6.4 How the Components Work Together

Figure 3.4 shows a simplified picture of how such an installation is put together, and the points below follow the figure.

1. Each **IP camera** is connected by cable to an access **switch**, which can also supply its power through PoE.
2. Access switches are linked to a **core switch** inside the **network rack** through patch panels and patch cords.
3. The **server** is connected to the same switch network. It receives the camera streams, stores them and runs the services used by staff.
4. **Office computers** and self-service devices join through other switches so that authorized staff and visitors can use the services.

5. A **router or firewall** controls traffic between the local network and the outside world.
6. Every cable and port is **labeled and tested** so that faults can be found and repaired quickly.

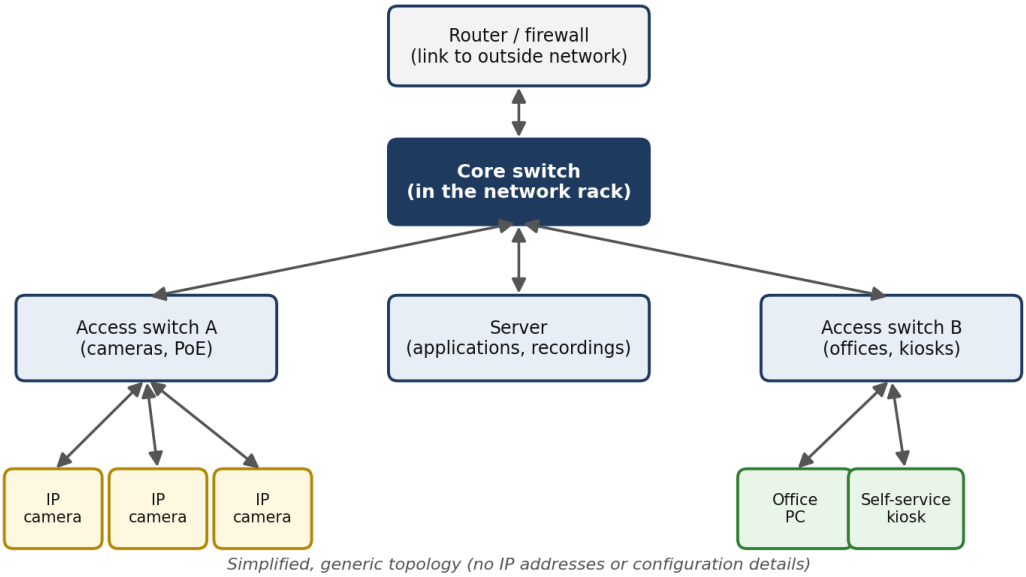


Figure 3.4 – Simplified topology of a network and security-camera installation like the one observed at MESOB (generic sample; edit to show only what you saw)

3.6.5 Installation Process Observed

- **Site survey and planning:** deciding the places for cameras, the rack and the cable routes.
- **Cable routing:** pulling data cables through ceilings, walls or trays from the rack to each point, away from power cables where possible.
- **Mounting:** fixing the rack, switches, server and cameras in place.
- **Termination and connection:** attaching connectors and plugging cables into patch panels, switches and devices.
- **Configuration:** setting up devices so that they can talk to each other
- **Testing:** checking each link, confirming that every camera shows video and that the server records it.

3.6.6 Photographs from the Visit

Figure 3.5 – [Server being configured during installation]



Figure 3.6 – [Monitoring screen showing several camera views]



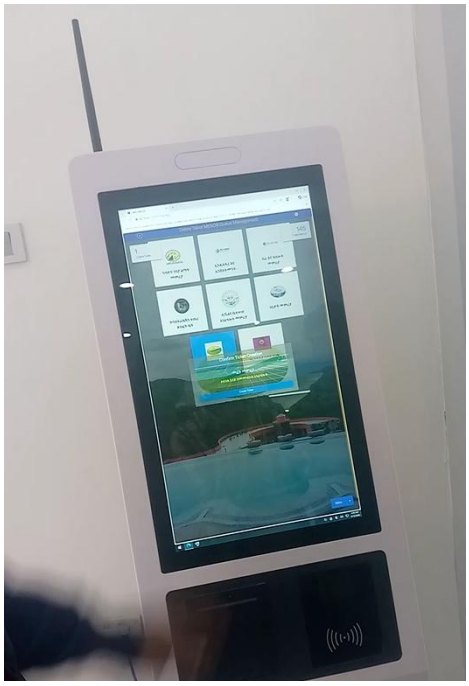
Figure 3.7 shows the reception area of the Debre Tabor MESOB service center. A lit wall sign with the name “Debre Tabor MESOB” and an Amharic line naming the one-stop service center of the City Administration stands above the reception desk. This is the public face of the initiative, where citizens come to receive services in one place.

Figure 3.7 – Reception area of the Debre Tabor MESOB service center



Figure 3.8 shows a self-service ticket kiosk with a large touchscreen. The screen shows a menu of service icons and a “Confirm Ticket” message, and below the screen there are a slot for a printed slip and a card-reader area. A device like this needs a constant network connection to the center’s servers, so it depends on the same network infrastructure that I saw being installed and configured.

Figure 3.8 – Self-service ticket kiosk with a touchscreen in the MESOB service center



In a rack with so many cables, clear labeling and tidy cable management are essential, because they make later maintenance and fault finding much easier.

3.6.7 Monitoring of the Security Cameras

At the site I also learned how the video of many cameras is shown together in a monitoring room, often called a situation room, where one screen displays several views at once and the recordings are kept on the server. This showed me the purpose of the whole installation: cameras, switches and servers are connected so that the people who manage a building can follow events in many places and keep the recordings

3.6.8 Good Practice Observed on Site

I noticed several good practices: following the site rules and wearing suitable protective clothing; using ladders and tools safely when working at height; keeping cables tidy and labeled; following the plan instead of improvising; testing each connection before moving on; and keeping the work area clean. These habits are as important in infrastructure work as technical skill.

3.6.9 Link to My Academic Knowledge and to the PMS

The visit connected my networking courses to real equipment. I saw LANs, switching, IP addressing, client–server communication, and the importance of reliability and security. I

also learned that cable quality, labeling and a neat rack have a big effect on how easy a network is to maintain.

The two parts of the internship are connected. The PMS runs on a server and is reached by browsers over a network, and its Socket.IO chat depends on a stable connection. Understanding how a local network, a server and its clients are arranged helped me think about how the PMS would be deployed in a pharmacy, how several computers (counter, store, cashier) would reach it, and how the server and its backups should be protected.

For security reasons, sensitive details of the installation, such as IP addresses, passwords and exact configurations, are not included in this report.

3.7 Major Challenges and Problems Faced

During the work I faced the following main challenges.

1. **Keeping stock consistent during a sale.** If the sale record is saved but the stock update fails (or the other way round), the data becomes inconsistent. Two cashiers selling the same medicine at the same time could also cause negative stock.
2. **Handling different payment methods.** Cash, bank, Telebirr and an online gateway behave differently. Some are confirmed instantly, others need manual confirmation.
3. **Changing the database while the system is in use.** New requirements (for example the `payment_method` and `status` columns) appeared after the first tables had been created, and recreating the database would delete existing data.
4. **Real-time chat reliability.** Messages had to appear immediately, be stored permanently, notify the right pharmacist, and show unread counts correctly.
5. **File upload handling.** Medicine and prescription images needed size and type control, safe storage and correct public paths.
6. **Dashboard calculations.** Weekly, monthly and yearly totals needed to follow calendar periods and not just “last 30 days”, and the calculation had to stay fast as data grows.
7. **Email sending from the server.** Gmail does not accept the normal account password for SMTP when two-step verification is active, so replies initially failed.
8. **Security of secrets and access.** API keys, the JWT secret and the database password had to be protected, and every route had to be limited to the right roles.
9. **Users who were new to digital tools.** Some staff were used to paper work and found the system unfamiliar at the beginning.

10. **Limited resources and time.** Limited internet access, a limited testing environment, and a fixed internship period made it necessary to prioritize features.
11. **Limited hands-on experience with network equipment.** I had studied networks in class, but I had never seen a real rack, patch panel or IP camera installation.
12. **Limited equipment and an informal organization at the Center.** There were few spare computers for testing, and tasks were agreed by discussion rather than by a written plan or tracking tool.

3.8 Measures Taken to Overcome the Challenges

Table 3.7: Challenges and measures taken

Challenge	Measure taken
Stock inconsistency	Used MySQL transactions so that the sale, its items and the stock decrement are committed together or rolled back; validated available quantity before checkout
Different payment methods	Added a `payment_method` VARCHAR column so any bank name can be stored; separated instant cash, manual bank/Telebirr confirmation and the Chapa online flow
Schema changes	Added automatic migration at server startup that adds missing columns to existing tables
Real-time chat	Used Socket.IO rooms per patient, stored every message in MySQL, loaded history on room join, and added toast notifications and unread badges
File uploads	Used Multer with separate folders for medicines and prescriptions and served them through the `/uploads` static path
Dashboard accuracy and speed	Computed all totals with MySQL queries in the backend using calendar-based date ranges and returned only the results
Email failure	Switched to a Gmail App Password stored in `EMAIL_PASS` and enabled two-step verification as required by Google

Challenge	Measure taken
Security	Hashed passwords with bcrypt, protected routes with JWT and role middleware, stored secrets in `.env`, kept `.env` out of Git and provided `.env.example`
User resistance	Held short demonstrations, used simple screens with color-coded badges and toast messages, and corrected the interface according to user feedback
Limited time	Prioritized core modules (inventory, POS, payment) first, then added chat, feedback and reports in later iterations
Little hands-on network experience	Asked the engineers and technicians questions on site, observed carefully and read about each device after the visit
Limited equipment and informal organization at the Center	Used free, open-source tools, agreed weekly goals with the supervisor, and kept my own task list and notes

3.9 Results and Discussion

3.9.1 Results Achieved

At the end of the internship, a working Pharmacy Management System had been developed with the following results.

- **Authentication and access control** work for Admin, Pharmacist and Cashier roles, and audit logs record user actions.
- **Inventory management** supports medicine records with images, categories, stock-inward, wastage, low-stock alerts and expiry alerts.
- **POS and payment** allow a cart-based sale with stock validation, automatic stock decrement, printable receipt and recorded payment method.
- **Prescription workflow** allows upload, approval and rejection with status tracking.
- **Real-time chat** connects public patients and pharmacists, with notifications and stored history.
- **Public catalog** displays live medicines with availability status.
- **Feedback inbox** allows the Admin to read messages and reply by email.
- **Dashboards and reports** show sales by period, expenses, profit, sales trend and daily cashier reports.

Figure 3.9 – Login page and Admin dashboard

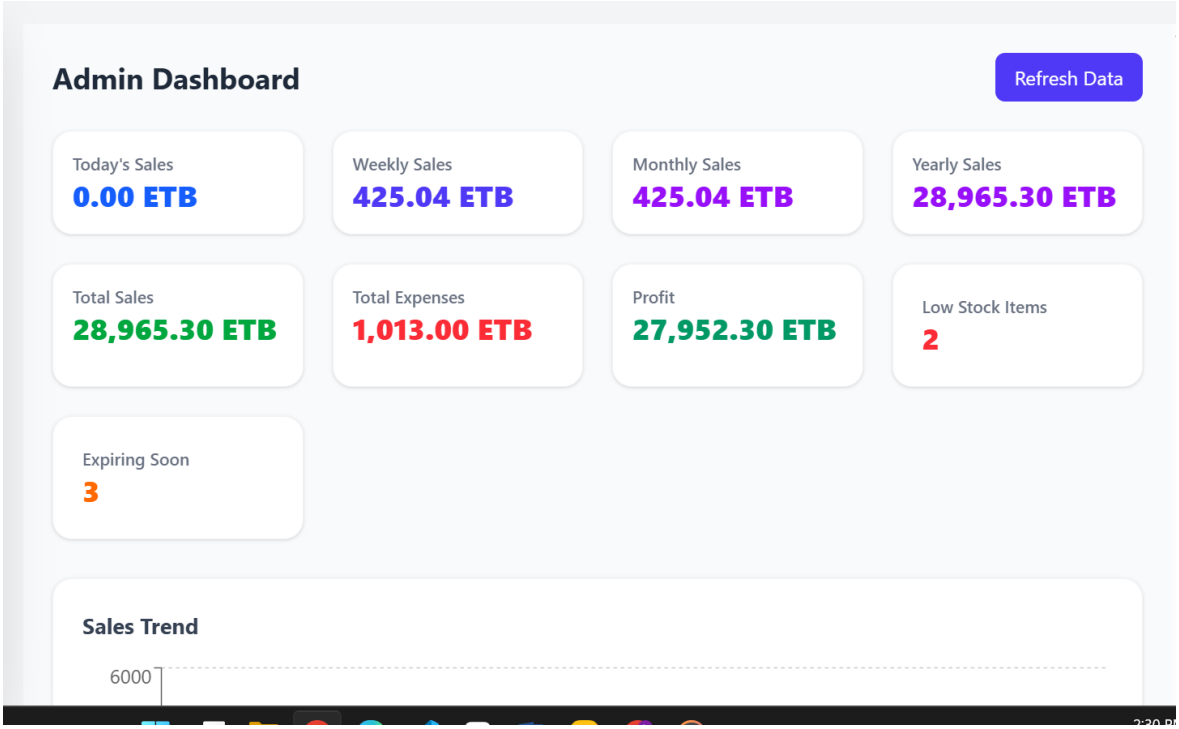


Figure 3.10 – Point-of-sale interface

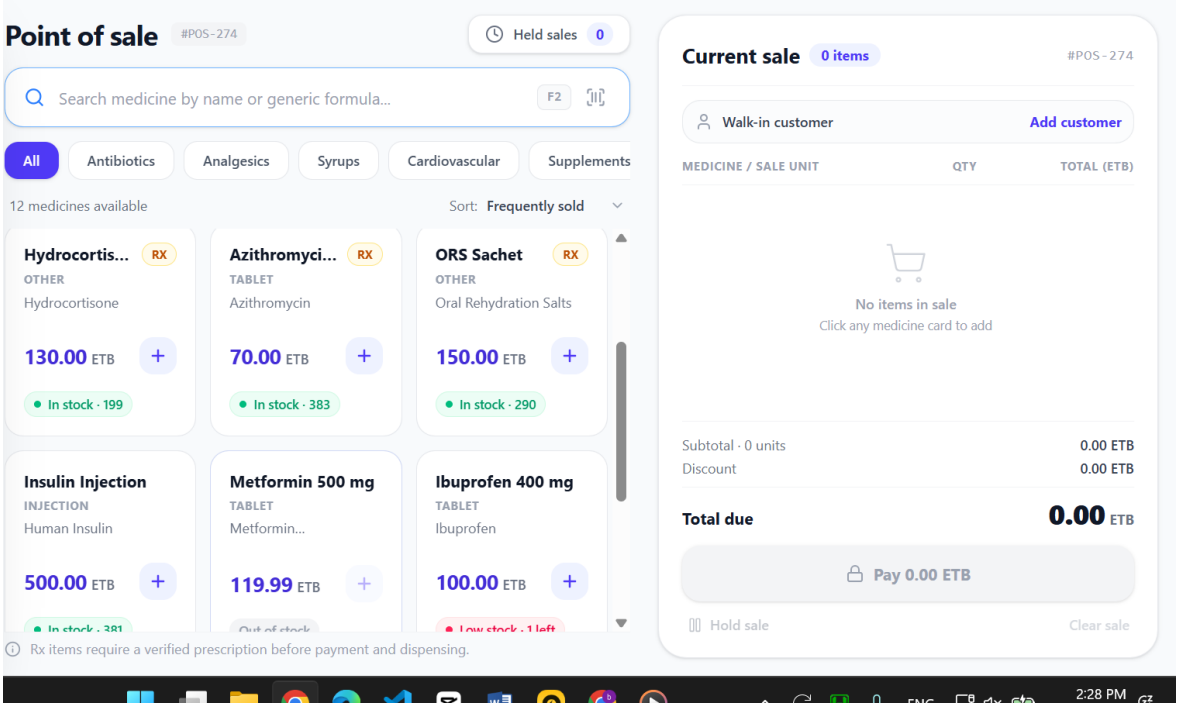


Figure 3.11 – Medicine inventory page

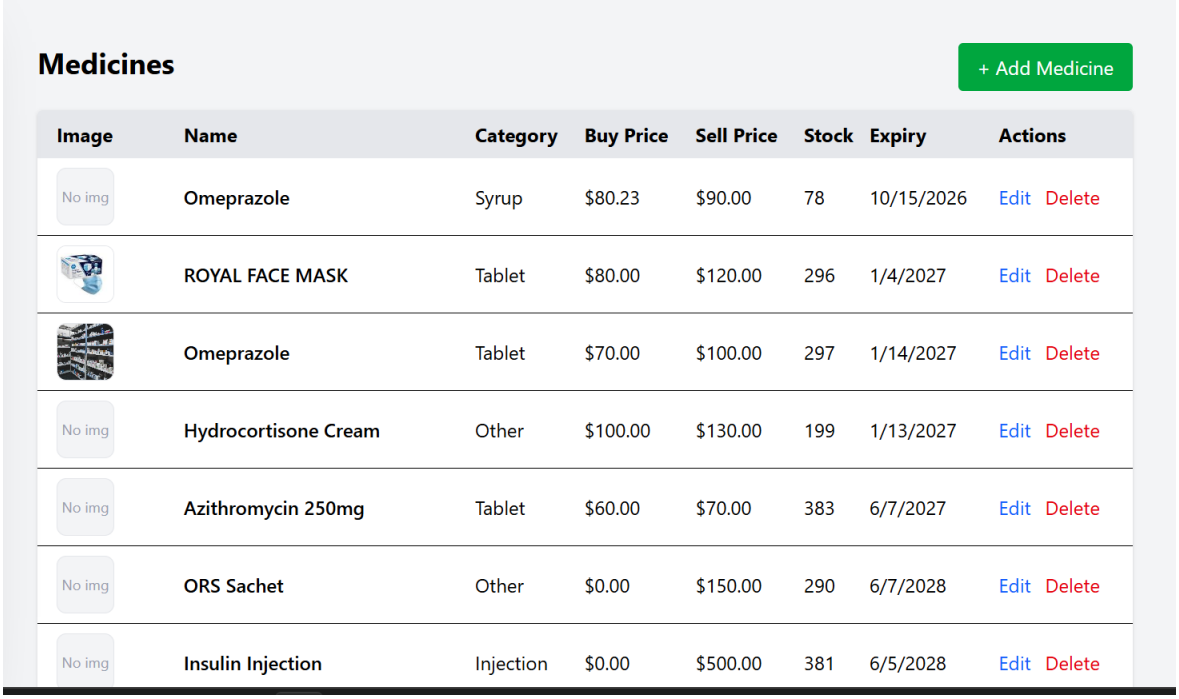


Figure 3.12 – Real-time patient chat

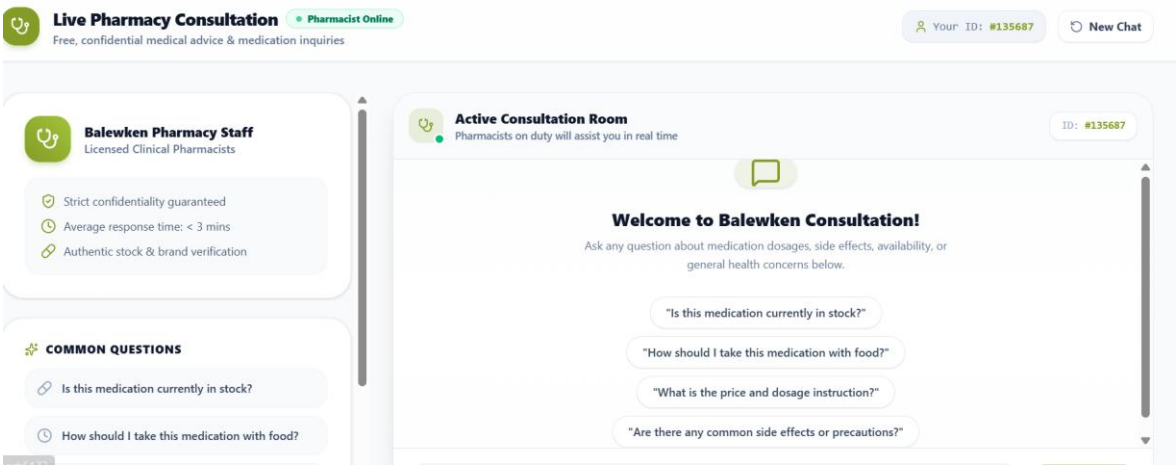


Figure 3.13 – Public product catalog

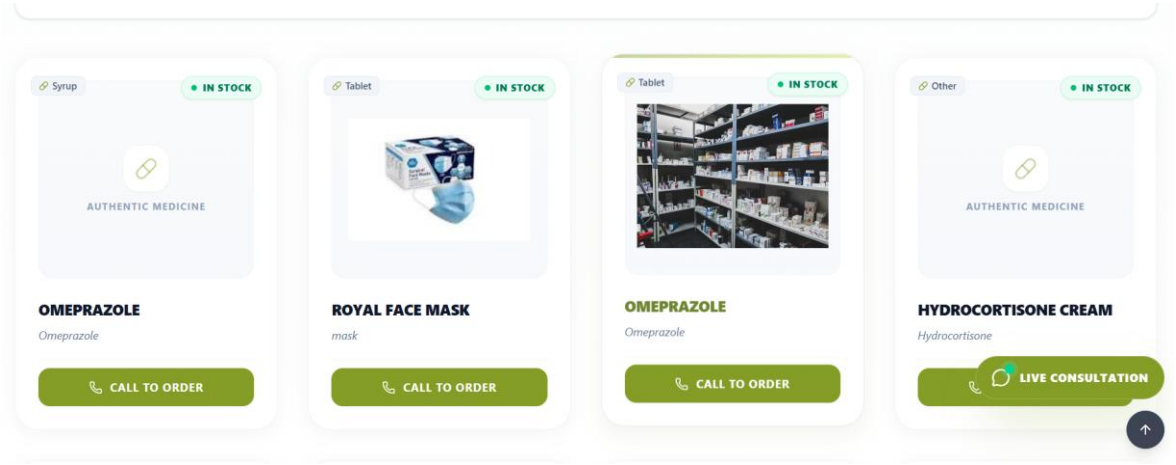


Table 3.8: Main API endpoints

Module	Method and endpoint	Description	Access
Auth	POST `/api/auth/register`	Register a new user	Public
Auth	POST `/api/auth/login`	Login and get JWT	Public
Auth	GET `/api/auth/me`	Get current user	Protected
Medicines	GET `/api/medicines`	List all medicines	Protected
Medicines	POST `/api/medicines`	Add medicine with image	Pharmacist
Medicines	PUT `/api/medicines/:id`	Update medicine with image	Pharmacist
Medicines	DELETE `/api/medicines/:id`	Delete medicine	Admin
Medicines	GET `/api/medicines/low-stock`	Low-stock list	Protected
Medicines	GET `/api/medicines/expired`	Expired list	Protected
Sales	POST `/api/sales`	Create sale with `payment_method`	Protected

Module	Method and endpoint	Description	Access
Sales	GET `/api/sales`	List sales (filter by cashier, date, status)	Protected
Sales	GET `/api/sales/:id`	Sale with items	Protected
Payments	GET `/api/payments/stats`	Dashboard KPIs	Protected
Chat	GET `/api/chat/conversations`	Active patient rooms	Pharmacist
Chat	GET `/api/chat/history/:roomId`	Message history	Pharmacist
Contact	POST `/api/contact`	Submit feedback	Public
Contact	GET `/api/contact`	List feedback	Admin
Contact	POST `/api/contact/reply`	Reply by email	Admin
Contact	PATCH `/api/contact/:id/read`	Mark as read	Admin
Contact	DELETE `/api/contact/:id`	Delete message	Admin
Other	`/api/users`, `/api/suppliers`, `/api/categories`, `/api/inventory`, `/api/expenses`, `/api/audit-logs`, `/api/prescriptions`, `/api/prescription-requests`	Resource management for each module	Role-based

3.9.2 System Testing

Testing was carried out module by module. The test cases below show the main scenarios;

Table 3.9: Test cases

ID	Test scenario	Input	Expected result	Actual result
TC-01	Login with valid account	Correct email and password	User enters the dashboard of his role	[Pass / Fail]

ID	Test scenario	Input	Expected result	Actual result
TC-02	Login with wrong password	Valid email, wrong password	Error message; no token issued	[Pass / Fail]
TC-03	Access admin resource as pharmacist	Call `/api/users` with pharmacist token	Access denied	[Pass / Fail]
TC-04	Add medicine with image	Valid form and image file	Medicine saved; image visible in list	[Pass / Fail]
TC-05	Low-stock alert	Quantity below threshold	Medicine appears in low-stock list	[Pass / Fail]
TC-06	Expiry alert	Expiry date near or past	Medicine appears in expiry alert	[Pass / Fail]
TC-07	Sell more than available stock	Quantity greater than stock	Sale refused with a message	[Pass / Fail]
TC-08	Normal cash sale	Valid cart, payment = cash	Sale saved; stock decreased; receipt printed	[Pass / Fail]
TC-09	Bank/Telebirr sale	Payment method = bank name or Telebirr	Method saved and shown with color badge in daily report	[Pass / Fail]
TC-10	Prescription medicine sale	Cart with prescription-required item	Prescription upload requested	[Pass / Fail]
TC-11	Approve prescription	Pharmacist clicks approve	Status changes to approved	[Pass / Fail]
TC-12	Patient chat message	Patient sends a message	Pharmacist gets toast and unread badge; message stored	[Pass / Fail]

ID	Test scenario	Input	Expected result	Actual result
TC-13	Chat history	Pharmacist opens old room	Earlier messages loaded from database	[Pass / Fail]
TC-14	Out-of-stock catalog item	Medicine with quantity 0	Out of Stock badge; Call to Order disabled	[Pass / Fail]
TC-15	Feedback reply	Admin replies to a message	Reply saved and emailed to customer	[Pass / Fail]
TC-16	Dashboard totals	Existing sales and expenses	Correct totals and profit shown	[Pass / Fail]
TC-17	Audit log	Perform create, update and delete actions	Actions appear in the log with user and time	[Pass / Fail]

3.9.3 Discussion

The results show that the main objectives of the project were met. Compared with the manual process, the system provides the following improvements.

Table 3.10: Comparison of existing and proposed system

Aspect	Existing manual system	Proposed PMS
Stock update	Manual, error prone	Automatic on every sale and stock-inward
Expiry follow-up	Depends on staff memory	System alerts
Reports	Hand calculated, slow	Instant dashboards and daily reports
Accountability	None	Audit logs for every action
Prescription control	Paper, hard to retrieve	Stored images with status
Payment records	Inconsistent	Payment method stored per sale

Aspect	Existing manual system	Proposed PMS
Customer contact	Only by visit or phone	Public catalog and live chat

The use of database transactions and stock validation addresses the most important technical risk, which is inconsistent stock. Storing the payment method as text rather than a fixed list made the system flexible for different Ethiopian banks and mobile-money services. The real-time chat adds a service that the manual system could not provide at all. Computing dashboard values in MySQL keeps the frontend light and the numbers consistent for all users.

The main limitations are:

- Prescription images are verified visually by the pharmacist; there is no automatic text recognition.
- Uploaded files are stored on the server's disk, which needs backup and does not scale well to many servers.
- The Chapa flow depends on internet access and on a correctly configured webhook.
- [Add any other limitation that you observed, such as no mobile application, no barcode scanner or no multi-branch support.]

3.9.4 Results of the Networking Exposure

The visit to the MESOB installation site gave me a clear picture of how servers, switches, cabling and security cameras are physically installed, configured and joined into one system, and of how planning, labeling and testing make a network reliable and easy to maintain. I also saw how many people must coordinate their work on such a site.

3.10 Recommendations Regarding the Identified Problems

Based on the experience gained, I recommend the following to the pharmacy or other organization that will use PMS, to solve the problems identified in manual pharmacy operation. Recommendations for the IoT Center itself are given in Chapter Five.

1. **Adopt PMS for all daily operations** and stop parallel paper registers after a short transition period, so that the database becomes the single source of truth.
2. **Count the physical stock regularly** and compare it with the system quantity, especially during the first months, to correct initial data entry errors.

3. **Define reorder levels** for each medicine so that low-stock alerts give accurate and useful warnings.
4. **Follow expiry alerts every week** and move near-expiry medicines to the front or return them to suppliers where possible.
5. **Make prescription verification mandatory** for all prescription-only medicines and train staff to use the approve/reject workflow.
6. **Back up the database and the uploads folder** automatically every day and keep a copy outside the pharmacy.
7. **Train all users** and prepare a short user manual so that new employees can learn the system quickly.
8. **Protect accounts** by using strong passwords, giving each employee a personal account and reviewing the audit log regularly.
9. **Host the system on a reliable server with HTTPS** before production use, and use the live Chapa keys only in that environment.

Chapter Four: Overall Benefits Gained from the Internship

4.1 Improvement of Practical Skills

The internship turned classroom knowledge into working skill. I built a complete application from the first database table to the final dashboard, and I learned how to organize a real project: separating back end and front end, structuring controllers, models and routes, managing environment variables and handling errors in one middleware. I gained hands-on skill with React hooks and context, Axios, Tailwind CSS, Express, MySQL queries and transactions, file upload, JWT authentication, Socket.IO, Nodemailer and the Chapa gateway. Debugging real problems, such as a failing e-mail or an inconsistent stock value, taught me to read logs, isolate causes and test fixes carefully.

On the networking side, I became familiar with real equipment: servers, switches, racks, patch panels, cables and IP cameras, and with the order in which an installation is carried out. Seeing a rack full of labeled cables was very different from the simple diagrams in my textbooks.

4.2 Upgrading of Theoretical Knowledge

The project linked many courses that I had studied separately. Database design and normalization, software engineering and requirement analysis, web programming, information security (hashing, tokens and access control), and modular design all became concrete. The ACID properties of transactions stopped being a definition when I saw why they are needed if two people sell the same medicine. The principle of least privilege became clear when I decided which role may call which endpoint.

The MESOB visit did the same for computer networks. LANs, switching, IP addressing, client–server communication and the role of firewalls, which I knew from lectures, appeared as physical devices that are connected, powered, labeled and tested.

4.3 Improvement of Industrial Problem-Solving Capability

In industry, problems do not arrive in textbook form. I learned to begin with a vague complaint such as “our reports are late”, look for its real cause, propose a technical solution, build it and check whether it worked. I learned to split a large system into small deliverable modules, to prioritize by importance and risk, and to choose solutions that suit the time and

resources available, for example manual confirmation of Telebirr and bank payments next to an online gateway.

4.4 Improvement of Team-Playing Skills

I wrote most of the code myself, but the work depended on my supervisor, on the people who explained pharmacy routines to me, and, during the site visit, on the engineers and technicians at MESOB. I learned to listen to different needs, to accept feedback and to change my plan when someone showed me a better workflow. At the MESOB site I also saw how technicians, engineers and the client's staff must coordinate: one team's work depends on another's, and a delay or mistake affects everyone. This showed me that technical skill alone is not enough and that teamwork is part of professional quality.

4.5 Improvement of Leadership Skills

I was responsible for planning my tasks, setting weekly goals, reporting progress and deciding how to solve technical problems. This trained my self-management, time management and accountability. By observing Mr. Abebe Kinde, who gives clear tasks, explains his decisions, supports people who struggle and still keeps projects moving, I also saw what leadership in a technical team looks like. When I demonstrated the system, I had to guide the viewers and answer their questions, which is also part of leadership.

4.6 Work Ethics, Industrial Psychology and Related Issues

Working on a pharmacy system made me understand professional ethics. Medicine records, prescriptions and patient chats are sensitive, so confidentiality, honesty and responsibility are essential, and this is why I used access control, hashed passwords and audit logs. Visiting a government service project also showed me why security details of an installation must stay confidential. I learned punctuality, respect for rules and for colleagues of different ages and backgrounds, safe behavior on a work site, and the need to keep promises about deadlines.

Regarding industrial psychology, I saw how workload, change and fear of mistakes influence people's behavior, and I learned that a new system must be simple, supportive and introduced patiently.

4.7 Entrepreneurship Skills

The internship showed me the business side of technology. I learned how a pharmacy earns money, what its main costs are (stock, expenses, wastage and expired goods) and how better information can raise profit. Building the sales, expense and profit dashboard made these ideas concrete. I also saw how the Center identifies needs, plans projects and delivers solutions with limited resources, and that there is real demand in Ethiopia for affordable local software. The same system could be developed into a product for other pharmacies, with installation and support services, which gave me ideas and confidence for starting a small software business in the future.

4.8 Interpersonal Communication Skills

I improved my ability to communicate professionally: asking clear questions during observation and discussions, writing e-mails and reports, giving short demonstrations, explaining technical issues in simple language, and speaking politely with technicians and client staff during the site visit. Designing the pharmacist quick-reply templates and the customer feedback reply also made me think about how to answer customers respectfully and clearly.

Table 4.1: Summary of skills gained

Area	What I gained
Practical skills	Full-stack development with React, Express, MySQL and Socket.IO; exposure to real network equipment
Theoretical knowledge	Practical application of databases, networking, security and software engineering
Problem solving	Analyzing real problems and delivering working solutions in iterations
Teamwork	Cooperation with my supervisor and others; observing team coordination at the MESOB site
Leadership	Planning, self-management and lessons from my mentor
Ethics and psychology	Confidentiality, responsibility, safety and understanding user behavior

Area	What I gained
Entrepreneurship	Business understanding and product thinking
Communication	Questions, demonstrations, writing and customer communication

Chapter Five: Conclusion and Recommendations

5.1 Conclusion

The internship at the IoT Center of Debre Tabor University gave me a valuable chance to apply my academic knowledge in a working environment. The main project, the Pharmacy Management System, was designed and built as a full-stack web application with React 19, Node.js and Express 5, MySQL and Socket.IO. It covers authentication and role-based access, inventory and expiry control, a point of sale with several payment methods including Chapa, prescription review, real-time patient chat, a public product catalog, customer feedback with e-mail reply, financial dashboards and audit logs. Through the arrangement of the Center Leader, I also visited the MESOB installation site and saw how servers, switches, cabling and security cameras are joined into one working system.

The project addressed the main weaknesses of manual pharmacy work: inaccurate stock, unnoticed expiry, error-prone sales records, weak prescription control, slow reporting and no accountability. Compared with the manual process, PMS offers more accurate data, faster reports, safer dispensing and better customer service. The main technical challenges, such as keeping stock consistent, handling different payment methods and evolving the database, were solved with transactions, automatic migrations and careful design.

Beyond the technical result, the internship improved my practical and theoretical knowledge, problem solving, teamwork, leadership, communication and sense of professional ethics. It also showed me that software depends on reliable infrastructure, and that developers gain from understanding it. The combination of university study and real practice is, in my view, essential for a competent software engineer.

5.2 Recommendations for the IoT Center

Based on my experience, I respectfully suggest that the IoT Center consider the following.

1. **Continue developing the PMS** into a full product: pilot it in a real pharmacy, add automated tests, move to live payment settings, and add the extensions listed in Section 5.4.
2. **Provide more equipment and workspace,** such as computers for testing, IoT boards and sensors, and network devices for practice, so more projects and interns can be supported.

3. **Define clearer roles and a written work plan,** with a simple tool for tracking tasks, so that the Center's work does not depend on one person alone.
4. **Strengthen software engineering practice** through code review, automated testing, a standard project structure and shared documentation templates.
5. **Offer a structured internship program** with a short orientation, a clear plan, regular feedback and learning goals, and let interns take part in hands-on tasks in more than one area, including network installation.
6. **Share knowledge between the software and network work** through internal workshops and a shared project repository.
7. **Document network installations** with diagrams and configuration records, and plan monitoring and maintenance after delivery.
8. **Invest in staff training** in areas such as cloud deployment, DevOps and security.
9. **Strengthen cooperation** with university departments, local businesses such as pharmacies, and government offices so that student projects reach real users.

5.3 Recommendations for the University

- Increase practical, project-based content in web development, database transactions, software testing and cloud computing.
- Provide hands-on networking laboratories where students can work with real switches, cabling and cameras before the internship.
- Strengthen the link between the university, the IoT Center and industry so that placements match the interests of students.
- Prepare students in advance for professional skills: teamwork, communication, report writing and work ethics.

5.4 Future Work on the System

PMS can be extended in these directions: barcode scanning, batch-level and first-expired-first-out (FEFO) stock handling, supplier purchase orders and invoices, SMS notifications, multi-branch support, a mobile application, live Chapa keys with webhook monitoring, automatic backups, HTTPS deployment on a reliable server, and automated tests.

References

1. Debre Tabor University IoT Center. 2001. *Center profile and project documents.* Debre Tabor University. (Internal document)
2. Debre Tabor University. (n.d.). *Official website: about the university, vision, mission and values.*
3. Meta Platforms. (n.d.). *React documentation.* https://react.dev
4. OpenJS Foundation. (n.d.). *Express – Node.js web application framework.* https://expressjs.com
5. Oracle Corporation. (n.d.). *MySQL 8.0 reference manual.* https://dev.mysql.com/doc
6. Socket.IO. (n.d.). *Socket.IO documentation.* https://socket.io/docs
7. Tailwind Labs. (n.d.). *Tailwind CSS documentation.* https://tailwindcss.com/docs
8. Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). IETF.
9. Chapa Financial Technologies. (n.d.). *Chapa developer documentation.* https://developer.chapa.co
10. Nodemailer. (n.d.). *Nodemailer documentation.* https://nodemailer.com
11. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson.
12. Elmasri, R., & Navathe, S. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
13. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson.
14. Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7th ed.). Pearson.

Appendices

Appendix A – Installation and Running Guide

Prerequisites: Node.js v18 or higher, MySQL v8 or higher, npm v9 or higher.

1. **Database setup:** run `mysql -u root -p < backend/sql/schema.sql` to create the `pharmacy_db` database and its tables. The backend also adds new columns (for example `payment_method` and `status`) automatically at startup.
2. **Backend:** in the `backend` folder run `npm install`, copy `.env.example` to `.env`, edit the values and run `npm run dev`. The server runs on `http://localhost:5000`.
3. **Frontend:** in the `frontend` folder run `npm install` and `npm run dev`. The application runs on `http://localhost:5173`.

Appendix B – Environment Variables

Variable	Description
`PORT`	Backend port = 5000
`DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`	MySQL connection settings
`JWT_SECRET`, `JWT_EXPIRE`	Token signing secret and expiry
`BCRYPT_ROUNDS`	Password hashing rounds
`CHAPA_SECRET_KEY`	Chapa API secret key
`STRIPE_SECRET_KEY`	Stripe API secret key (optional)
`EMAIL_USER`, `EMAIL_PASS`	Gmail address and App Password for feedback replies

Appendix C – Project Folder Structure

Folder	Content
`backend/config`	`database.js` – MySQL connection pool
`backend/controllers`	Auth, medicine, sale, supplier, user, audit and prescription controllers

Folder	Content
backend/middleware	auth.js, errorHandler.js
backend/models	Medicine, Sale, Supplier, User
backend/routes	Route files for every module, including chat and contact
backend/sql	schema.sql
backend/uploads	medicines/ and prescriptions/ images
backend/server.js	Entry point, Socket.IO and auto-migration
frontend/src/components	Layout, Sales (POS and Receipt), currency formatter, public Home, Abouts, Product and contact components
frontend/src/context	AuthContext.jsx
frontend/src/pages	Admin, pharmacist, cashier, chat, main and login pages
frontend/src/services	api.js Axios instance

Appendix D – Interview / Observation Questions

1. How do you currently record medicine stock and sales?
2. How do you know when a medicine is nearly out of stock or about to expire?
3. How are prescriptions checked and stored?
4. Which payment methods do customers use, and how are they recorded?
5. How long does it take to prepare the daily or monthly report?
6. What mistakes or losses happen most often?
7. What would you like a new system to do for you?

Appendix E – Additional Screenshots and Weekly Activity Log

(Attach the extra screenshots of the system, the Center's internship attendance sheet and the weekly activity log signed by your supervisor. If you recorded a video at the MESOB site, give its file name or link here.)

Week	Activity	Supervisor's signature
1	Orientation at the IoT Center, observation of pharmacy work, requirement gathering	
2	Requirement analysis and system design	
3	Database design and authentication module	
4	Medicine and inventory module	
5	POS and sale module	
6	Payment methods and Chapa integration	
7	Prescription module	
8	Real-time patient chat	
9	Public catalog, feedback and email reply	
10	Dashboards, reports and audit logs	
11	Testing and bug fixing	
12	Documentation, user training and final report	

Site visit: August 13 visit to the MESOB network installation site of Debre Tabor City Administration, arranged by Mr. Abebe Kinde. Supervisor's signature: _____